

ТЕХНИЧЕСКОЕ ЗАДАНИЕ
на разработку веб-приложения пиццерии

Версия документа: 1.0
2026

СОДЕРЖАНИЕ

1. Термины и определения
2. Общие положения
 - 2.1. Назначение документа
 - 2.2. Цели создания системы
 - 2.3. Основные функциональные возможности системы
 - 2.4. Использование технического задания
3. Функциональные требования
 - 3.1. Действующие лица системы
 - 3.2. Диаграмма вариантов использования
 - 3.3. Описание вариантов использования
4. Требования к экранным формам
5. Модель данных
6. Нефункциональные требования
7. Требования к приемке-сдаче проекта

1. Термины и определения

1.1. Общие термины

Система — веб-приложение пиццерии, предназначенное для просмотра меню, формирования корзины, оформления и отслеживания заказов.

Пользователь — зарегистрированный посетитель системы с ролью user.

Администратор — зарегистрированный пользователь с ролью admin, имеющий дополнительные права для управления заказами и меню.

Заказ — набор выбранных пользователем пицц, оформленный для приготовления.

Меню — перечень доступных в системе пицц.

Корзина — временное хранилище выбранных пользователем пицц до оформления заказа.

Табло — отдельная страница системы, на которой отображаются заказы, готовые к выдаче.

1.2. Технические термины

Frontend — клиентская часть веб-приложения.

Backend — серверная часть приложения, обеспечивающая получение и изменение данных.

API — программный интерфейс взаимодействия клиентской части с сервером.

JSON Server — используемый в проекте сервер, предоставляющий REST API на основе файла db.json.

БД — хранилище данных приложения. В рамках текущей реализации используется файл db.json.

Zustand — библиотека управления глобальным состоянием приложения.

React — библиотека для разработки пользовательского интерфейса.

TypeScript — язык программирования с поддержкой статической типизации.

2. Общие положения

2.1. Назначение документа

Настоящее техническое задание определяет требования к разработке веб-приложения пиццерии.

Документ определяет состав реализуемых функций, требования к интерфейсу, данным, архитектуре, безопасности и приемке программного продукта.

2.2. Цели создания системы

- Предоставление пользователю удобного веб-интерфейса для просмотра меню пиццерии.
- Предоставление возможности зарегистрироваться и авторизоваться в системе.
- Предоставление возможности сформировать корзину из выбранных пицц.
- Предоставление возможности оформить заказ.
- Предоставление пользователю возможности просматривать историю собственных заказов.
- Предоставление администратору возможности управлять заказами.
- Предоставление отдельного табло для отображения готовых заказов.

2.3. Основные функциональные возможности системы

Для пользователя система должна предоставлять:

- регистрацию;
- авторизацию;
- просмотр меню;
- просмотр информации о пиццах;
- добавление пицц в корзину;
- удаление пицц из корзины;
- просмотр итоговой стоимости;
- оформление заказа;
- просмотр собственных заказов;

- просмотр статуса заказа;
- просмотр готовых заказов на табло.

Для администратора дополнительно должны быть доступны:

- просмотр всех заказов;
- изменение статуса заказа;
- загрузка изображения пиццы;
- доступ к административной панели.

2.4. Использование технического задания

Настоящее техническое задание используется как основной документ, определяющий состав и требования к системе.

Все изменения требований после утверждения документа должны быть согласованы сторонами.

При выявлении неоднозначностей требования должны быть уточнены до реализации соответствующей функциональности.

3. Функциональные требования

3.1. Действующие лица системы

1. Неавторизованный пользователь.
2. Пользователь.
3. Администратор.

Иерархия ролей: Пользователь → Администратор.

Администратор обладает всеми возможностями обычного пользователя и дополнительными правами управления системой.

В системе предусмотрены две роли: user и admin. Администратор создается при инициализации базы данных.

3.2. Описание вариантов использования

3.3.1. ВИ «Регистрация пользователя»

Неавторизованный пользователь должен иметь возможность создать учетную запись в системе.

Основной поток: открыть страницу регистрации; ввести имя, email и пароль; пройти проверку данных; проверить уникальность email; создать учетную запись с ролью user; автоматически авторизоваться.

При некорректных данных система должна показать сообщения об ошибках.

3.3.2. ВИ «Авторизация пользователя»

Зарегистрированный пользователь должен иметь возможность войти в систему по email и паролю.

После успешной проверки пользователь сохраняется в глобальном состоянии и получает доступ к защищенным разделам.

Неавторизованный пользователь не должен иметь возможности оформить заказ или просматривать историю заказов.

3.3.3. ВИ «Просмотр меню»

Пользователь должен иметь возможность просматривать список доступных пицц.

Карточка пиццы должна содержать изображение, название, список ингредиентов, цену и кнопку добавления в корзину.

3.3.4. ВИ «Работа с корзиной»

Пользователь должен иметь возможность добавить пиццу в корзину, удалить пиццу, просмотреть содержимое корзины и увидеть итоговую стоимость.

Итоговая стоимость должна автоматически пересчитываться после изменения содержимого корзины.

3.3.5. ВИ «Оформление заказа»

Пользователь должен быть авторизован.

При оформлении система формирует заказ с идентификатором, идентификатором пользователя, списком пицц, датой и временем создания и статусом.

После успешного сохранения заказа корзина очищается.

3.3.6. ВИ «Просмотр истории заказов»

Авторизованный пользователь должен иметь возможность просматривать только собственные заказы.

Для заказа отображаются состав, итоговая стоимость, статус, дата и время создания и время готовности, если заказ готов.

3.3.7. ВИ «Просмотр табло готовых заказов»

На табло должны отображаться заказы со статусом «Готово».

Для заказа отображаются номер заказа и имя клиента.

После перевода заказа в состояние «Выдан» он должен исчезнуть с табло.

3.3.8. ВИ «Управление заказами администратором»

Администратор должен иметь возможность просматривать все заказы и изменять их статусы.

Используются четыре статуса: «Ожидает», «Готовится», «Готово», «Выдан».

4. Требования к экранным формам

4.1. Форма «Регистрация»

Поле	Тип	Требование
Имя	Строка	Обязательное
Email	Строка	Обязательное, проверка формата
Пароль	Строка	Обязательное
Подтверждение	Строка	При необходимости
Кнопка регистрации	Кнопка	Выполняет регистрацию

4.2. Форма «Авторизация»

Поле	Тип	Требование
Email	Строка	Обязательное
Пароль	Строка	Обязательное
Войти	Кнопка	Выполняет авторизацию

4.3. Страница «Меню»

- изображение;
- название;
- ингредиенты;
- цена;
- кнопка «В корзину».

В исходной реализации меню содержит 18 первоначально загруженных позиций.

4.4. Страница «Корзина»

- список выбранных пицц;
- возможность удалить пиццу;
- итоговая стоимость;
- кнопка оформления заказа.

Если корзина пустая, оформление заказа должно быть недоступно.

4.5. Страница «Мои заказы»

- номер заказа;
- состав;
- стоимость;
- статус;
- дата создания;
- дата готовности, если имеется.

Пользователь не должен видеть заказы других пользователей.

4.6. Административная панель

Административная панель доступна только пользователю с ролью admin.

На странице должны отображаться все существующие заказы с возможностью изменения статуса.

4.7. Страница «Табло»

На странице отображаются только заказы со статусом «Готово».

- номер заказа;
- имя клиента.

После изменения статуса заказа данные на табло должны обновляться.

5. Модель данных

В текущей реализации используется файл `db.json`, содержащий три основные коллекции: `users`, `pizzas` и `orders`.

5.1. Сущность User

Поле	Тип	Описание
id	string	Уникальный идентификатор
name	string	Имя пользователя
email	string	Email для входа
password	string	Пароль
role	user/admin	Роль пользователя

5.2. Сущность Pizza

Поле	Тип	Описание
id	string	Уникальный идентификатор
name	string	Название
price	number	Цена в рублях
ingredients	string[]	Список ингредиентов
imageUrl	string	Путь к изображению

5.3. Сущность Order

Поле	Тип	Описание
id	string	Уникальный идентификатор заказа
user	User	Пользователь
pizzas	Pizza[]	Список пицц
status	OrderStatus	Статус
createdAt	Date	Дата и время создания
readyAt	Date?	Дата и время готовности

6. Нефункциональные требования

6.1. Требования к интерфейсу

- Интерфейс должен быть понятным и удобным для пользователя.
- Основные операции должны выполняться через понятные элементы управления.
- Ошибки ввода должны сопровождаться информативными сообщениями.
- Интерфейс должен корректно отображать страницы пользователя и администратора.
- Административные элементы не должны отображаться обычному пользователю.

6.2. Требования к программному обеспечению

Frontend приложения должен быть реализован с использованием React, TypeScript, Vite, CSS Modules, Zustand и React Router.

Для взаимодействия с серверной частью используется JSON Server, работающий с файлом db.json.

6.3. Требования к производительности

- Страницы приложения должны загружаться без существенных задержек.
- Запросы к серверу должны обрабатываться без зависания пользовательского интерфейса.
- После изменения данных интерфейс должен своевременно отображать актуальное состояние.
- Операции с корзиной должны выполняться без заметной задержки.
- Изменение статуса заказа должно отображаться после успешного ответа сервера.

6.4. Требования к безопасности

- Доступ к административной панели предоставляется только пользователям с ролью admin.

- Обычный пользователь не должен иметь доступа к административным функциям.
- Неавторизованный пользователь не должен иметь возможности оформить заказ.
- Неавторизованный пользователь не должен иметь доступа к истории заказов.
- При прямом переходе по URL защищенной административной страницы система должна проверять роль пользователя.

7. Требования к приемке-сдаче проекта

Для приемки проекта должны быть предоставлены:

- исходный код системы;
- рабочая версия веб-приложения;
- файл db.json;
- техническое задание;
- документация по запуску проекта;
- тестовые сценарии.

Приемо-сдаточные испытания должны включать проверку основных пользовательских сценариев.